Task Aware Dynamic Resource Allocation in Industrial Private 5G Networks

Afreen Haider
ahaider@ucsd.edu

Soham Guha Mazumder
smazumder@ucsd.edu

Aditya Mallick
admallick@ucsd.edu

***Abstract*—Industrial automation under Industry 4.0 imposes stringent requirements on ultra-reliable low-latency communication (URLLC), task-aware scheduling, and real-time safety guarantees. Public 5G networks are not optimized for mission-critical robotic coordination in confined industrial environments, motivating the deployment of private 5G systems with full control over radio resource management. This report presents the design and simulation of a task-aware dynamic resource allocation framework for an industrial private 5G network supporting autonomous industrial robots.The proposed system integrates collision probability prediction using kinematic modeling with adaptive resource block (RB) allocation and modulation and coding scheme (MCS) selection. A 10 MHz OFDM-based downlink with 7 resource blocks is implemented, incorporating industrial channel models. Collision risk is estimated via future position prediction and smoothed using an exponential moving average. Based on both task priority and predicted collision probability, the base station dynamically reallocates RBs and adapts MCS between QPSK, 16-QAM, and 64-QAM to balance throughput and robustness. The framework is evaluated in a factory-scale simulation using SUMO and TraCI, modeling two industrial robots executing heterogeneous tasks with varying priority levels. Results demonstrate that the proposed sensing-to-action pipeline successfully prevents collision events while maintaining higher throughput for priority tasks. Adaptive MCS switching ensures reliable actuator command delivery under fading conditions, and velocity traces confirm correct decoding and execution of downlink control commands. Finally, we implement RL-based scheduling and extend that to the 5-robot case to show an improvement in performance over the heuristic model.**

***Index Terms*—Private 5G, Downlink URLLC, Resource Block, Industrial Robots, Dynamic Reinforcement Learning**

I. Introduction

A. Private 5G networks

A private 5G network is a dedicated 5G network with enhanced communication protocols, optimized services, and context-based implementation. Private 5G networks can be independently managed by their owners, who can control every aspect of the network, such as priority scheduling and resource allocation. Within a defined application area, (e.g., an industrial complex or plant), the private 5G network serves a multitude of devices, user equipment or industrial components. In order to offer services, a private 5G network might subsume some of the capabilities of public 5G networks. Consequently, 3GPP has defined two implementations for private 5G networks - stand-alone deployment and public network integrated deployment. In the standalone deployment, all network functions are restricted to operate within the

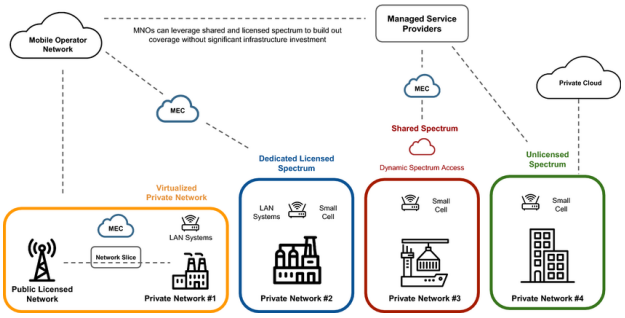


Fig. 1: Private 5G Network Architecture as visualized by Private 5G networks: a survey on enabling technologies, deployment models, use cases and research directions [6]

defined regions, and the network is allocated private spectrum within the Industrial, Scientific, and Medical (ISM) band. The radio access network (RAN) and the core network may be isolated from the public (stand-alone) or might be shared with the public network (public network integrated deployment). Consequently, compared with a stand-alone deployment, this deployment has lower customization, self-control, and security. Several KPIs need to be taken care of while analyzing a private 5G network deployed in an industrial factory setting with multiple industrial robots (IRs) and one private 5G base station.

Ultra-reliable low-latency communications (URLLC), massive connectivity, and specialized task-aware resource allocation can be achieved by deploying a private 5G base station with custom features. IRs often work in mission-mode in critical factory settings, where even a small error, caused by a deep fade when the IR line-of-sight (LoS) is blocked by a wall, can cause massive revenue losses. A private base station allows us to tailor our network to optimize our specific industrial context.

B. Motivation and Contributions

Industry 4.0 is expected to follow stringent KPIs on reliability, URLLC, and massive IR connectivity, RAN flexibility, and intelligent resource allocation for optimizing mission-mode throughput. Public 5G networks are primarily developed for the general user population, incorporating user fairness and relatively relaxed QoS requirements. An industrial setting requires task-aware decisions being made at lightning speed with a high reliability index. Certain tasks have a higher priority and more resources should be allocated for those IRs. Moving IRs

have a risk of collision, reducing overall performance output. A private 5G base station monitors each IRs location and calculates the inter-IR distance and the corresponding collision probability, and can subsequently inform the IR to slow down. Clearly, the collision distance threshold should be a function of the underlying factory size, the modulation order, and the velocity of the IR. In our implementation, we implement a private 5G network with dynamic resource allocation for task-aware operation and collision avoidance. At each time step, the base station senses the precise IR coordinates and velocity using either a bi-static or mono-static sensing model. Thereafter, it calculates the collision probability profile across all IR pairs and reduces the modulation and coding scheme (MCS) to a robust QPSK from a high data rate 64-QAM. We quantify the performance of our dynamic resource allocation through latency, throughout and spectral efficiency, and plots like resource block allocation over time, IR speed versus actuation command, and collision probability across simulation time.

C. Related works

The authors of [4] develop a comprehensive overview of the field of private 5G, presenting its different deployment types like stand-alone and public network assisted deployment. Performance comparisons were made to WiFi 6, and it was concluded that low cost of deployment was the only positive feature of WiFi 6 compared to private 5G. The authors of [3] divided industrial traffic contexts into 4 different logical slices via vertical network slicing. An algorithm for hierarchical scheduling was developed where control traffic or high priority traffic is given a different logical network (L1) as compared to simple monitoring traffic (L3). The paper assigns tasks with different priority order, and assumes a stream of priorities being sent to the RAN which reconfigures resources. We build on this by actually generating the mapping between the tasks and the priority order as well.

A reinforcement learning (RL) approach was studied in [5] to prevent handoffs during critical workloads. The algorithm locks connection to a specific base station when it detects a mission critical workload under process, in order to prevent hand-off jitter in the robot. This idea proved that the RL algorithm can learn from the robot's routine pattern and the inherent statistical consistency of the robot channel states, to make an accurate decision. However, the paper does not jointly model the collision risks between multiple robots. Traditional scheduling algorithms like proportional fairness (PF) struggle in dynamic industrial scenarios especially in B5G bands like mmWave. Building on this, [2] proposed an Enhanced-PF algorithm that adapts more rapidly to channel fluctuations . However, their approach depends on reactive metrics (CQI) instead of proactive Sensing-to-Action. A deep RL framework was proposed by [1] Learn to Schedule (LEASCH): A Deep Reinforcement Learning Approach for Radio Resource Scheduling in the 5G MAC Layer, to replace heuristic resource allocation rules by balancing throughput and fairness dynamically. However, LEASCH focuses on metrics like buffer size and SNR, and does not account for physical collision avoidance as a cause for dynamic resource allocation.

II. System Model

The system model is implemented in simulation using the Simulation of Urban Mobility(SUMO) open source software. The scenario is designed as a factory/warehouse environment with multiple task benches and obstacles and two autonomous robots moving in the environment performing industrial tasks. Non environment aware movement of the robot is already integrated into their systems. A 5G network specifically designed for the environment communicates with the robots to help prevent collisions and manage movement.

The communication channel between the robots and the base station is divided into three phases - an uplink component, base station processing and the down-link component for the base station to pass on the actuation command to the robot. The communication channel is modeled as an OFDM based network using 7 resource blocks (84 subcarriers) with a total bandwidth of 10 MHz. The base station gets the x and y coordinates of the robots from the environment, computes the collision probability between robots and based on the algorithm/rule specified allocates a given number of resource blocks in the network to communicate the actuation command to the robot.

The allocation of resource blocks follows a rule that is governed by the priority number of a given robot. The priority number of a robot has two components - collision priority and task priority.

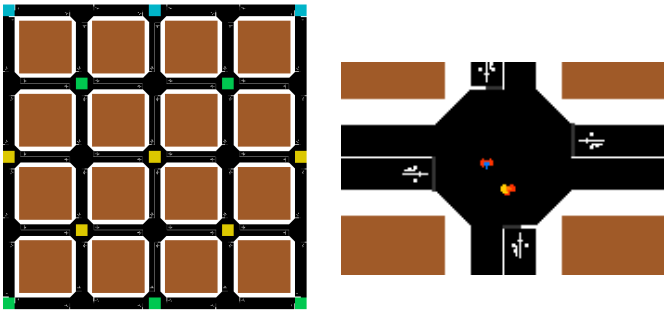
We refer to the private 5G base station as ***BS*** and each robot as ***IR*** (Industrial Robot).

A. SUMO - GUI

Simulation of Urban Mobility is an open source software which we employ to simulate the industrial factory/warehouse environment. It allows us to model vehicles and interactions between the vehicles including lane-changing behavior which is crucial to prevent collisions in the environment.

1) Factory environment: The industrial factory environment is modeled as a 200 m $\times$ 200 m with 13 workstations manned across a 5 $\times$ 5 grid. Workstations are the destination locations of the *IRs*. The environment model with the robots are shown in 2a The priority of the task being executed at a workstation is indicated by its color - `GREEN` stands for the highest priority, `YELLOW` for medium priority and `CYAN` for lowest priority. The brown squares are the workstations racks which are essentially obstacles for the *IRs* to maneuver around.

2) Industrial Robots: The initial simulation uses two Industrial robots modeled as the smallest `SUMO` GUI unit as shown in 2b. The two *IRs* are initialized at opposite corners of the factory grid and tasks are assigned to each *IR* using a random generator. The simulation uses `SUMO`'s inbuilt routing algorithm that calculates the best path for the *IR* using Dijkstra's algorithm. This structuring calculates the shortest path for edge-to-edge routing instead of node-to-node due to the restricted movements of the *IRs*. Five types of tasks are allocated to the *IRs* their priorities are listed in table I



(a) Factory simulation model (b) Industrial Robots

Fig. 2: SUMO simulation set-up

Task	Priority
Loading	3
Processing	2
Assembly	1
Storage	2
Inspection	3

TABLE I: Industrial Robot Tasks and respective priority levels

3) BS Interface: The base station for the private 5G network is located in the center of the warehouse at a height of 5m $(100, 100, 5)$ where the robots are at ground level. We use `TraCl` (Traffic Control Interface) python API to control and retrieve data from the SUMO simulation. Using `TraCl` we can extract the x and y coordinates of each *IR* and send the information to the base station.

B. Base-station and down-link architecture

Base station reads information about each IR at time intervals of 0.1 s. After reading the data, the base station computes collision probability between each IR pair, decides the amount of resource blocks (RBs) to allocate to each IR, selects a modulation scheme for the transmission, builds an OFDM downlink packet with a control command, transmits it through the simulated channel and uses the `TraCl` API to actuate the robot based on the given command.

1) Collision probability: The collision probability between each pair of industrial robots (only a single pair for the first stage of testing) is computed using kinematic prediction. We assume that at each time step of the BS receiving data from the `TraCl` API, the coordinates and the velocity of each IR are computed perfectly so we can use a constant velocity model to predict the collision probability.
In the scenario with 2 *IRs*, at time step $t = \tau_0$, the projected positions of the two *IR*s at time step $t = \tau_0 + \Delta\tau$ can be written as

$$p_{0,\text{future}} = (x_0 + v_{x0} * \Delta\tau, y_0 + v_{y0} * \Delta\tau) \quad (1)$$

$$p_{1,\text{future}} = (x_1 + v_{x1} * \Delta\tau, y_1 + v_{y1} * \Delta\tau) \quad (2)$$

Here (x_0, y_0) and (x_1, y_1) are the coordinates of each of the *IRs* and (v_{x0}, v_{y0}) and (v_{x1}, v_{y1}) are their velocities taken from the API at time step τ_0. Predicated distance of collision will be calculated as

$$d_{\text{pred}} = ||p_{0,\text{future}} - p_{1,\text{future}}|| \quad (3)$$

Collision probability is computed using the formula

$$P_c = max(0, 1 - \frac{d_{\text{pred}}}{d_{\text{thres}}}) \quad (4)$$

d_{thres} is the threshold distance between the two *IRs* which is tunable according to the industrial setting.
Because of calculating the probability of collision in time steps of $\Delta\tau$, the actuation commands provided to the *IRs* become jittery. To prevent this, we smoothen the collision probability by using a exponential moving average with multiplier α chosen as 0.3

$$P_{c,\text{smooth}} = \alpha \cdot P_c(\tau_0) + (1 - \alpha) \cdot P_c(\tau_0 - 0.1) \quad (5)$$

$P_c(\tau_0)$ is the collision probability computed according to equation (4) at time step τ_0

2) Resource Block Allocation: We employ a 10 MHz bandwidth network which is divided into 7 resource blocks each containing 12 subcarriers. The base split of the resource blocks between the two *IRs* is based on their task priority levels p_0 and p_1

$$rb_0 = \text{floor}\big(7 \times \frac{p_0}{p_0 + p_1}\big) \quad (6)$$

$$rb_1 = 7 - rb_0 \quad (7)$$

The *IR* that has higher task priority will be allocated more resource blocks for its communication channel with the *BS*. Allocating more RBs implies higher bandwidth and therefore more reliable communication to transmit actuation information to the *IR*.

Detecting a collision The system detects a possible future collision if $P_c > 0.5$. The system allocates resource blocks to the *IRs* involved in the collision as evenly as possible, giving the ceiling half to the higher-priority *IR* to preserve its lower latency advantage.

$$rb_0 = 4, \quad rb_1 = 3 \quad (\text{for } p_0 \geq p_1) \quad (8)$$

At the time of predicted collision, the task priority component in allocating resource blocks is overridden by the priority of collision.

3) Modulation Coding Scheme: The modulation and coding scheme (MCS) for each transmission with the *IR* is decided based on the collision probability and the available SNR of the downlink channel according to table II A higher collision risk would require the system to transfer the actuation command with less error probability. At higher SNR, the error probability is guaranteed to be lower hence we can prioritize a faster modulation coding scheme.

Condition	MCS	Robustness
$P_c > 0.6$ OR SNR < 15 dB	QPSK	Robust
$0.3 < P_c \leq 0.6$	16-QAM	Moderate
$P_c \leq 0.3$	64-QAM	Fastest

TABLE II: MCS Selection

Probability	Command
$P_c \geq 0.65$	`STOP`
$P_c \geq 0.3$ AND $P_c < 0.65$	`SLOW`
else	`CONTINUE`

TABLE III: Actuation commands

4) Actuation commands: If no collision is detected for the *IRs*, the base station sends the command `CONTINUE` to let the *IR* continue the path it is taking and the task being executed. When a collision is detected, the commands sent by the *BS* based on the collision probability threshold is mapped in table III

5) PHY Channel: The payload for the downlink channel is a 2-bit actuation command extended with 8 bits for a Cyclic Redundancy Check (CRC) and 150 zero-padded bits, giving 160 bits in total. In the MCS allocation, we have 3 options — QPSK maps 160 bits to 80 QAM symbols, 16-QAM to 40 QAM symbols, and 64-QAM to 27 QAM symbols. Since each OFDM time symbol carries $n_{\text{RB}} \times 12$ subcarriers, the number of OFDM symbols required per packet is $n_{\text{sym}} = \lceil n_{\text{QAM}}/(n_{\text{RB}} \times 12) \rceil$, meaning the RB allocation directly determines per-*IR* packet latency. The symbols are sent on the designated subcarriers of the allocated resource blocks. Remaining subcarriers are either utilized as guard band or unused. A 64-point IFFT is performed at the *BS* to create the time domain signal and a 16 length cyclic prefix is added to make the total number of samples for a symbol equal to 80.

The channel model is divided into 3 components - **Pathloss** for the down-link channel is modeled according to the equation (9)

$$L_{dB} = 38 + 30 \times \log_{10}(d_{\text{3D}}) \tag{9}$$

where d_{3D} is the three dimensional distance between the base station and the *IR*.

Small scale pathloss is modeled according to **Rayleigh fading** as a single-tap complex Gaussian. Assuming the velocity of the *IR* to be 5 m/s, and using a center frequency of $f_c = 3.5$ GHz, the coherence time of the channel can be calculated as

$$T_c = \frac{9}{16\pi f_d} \tag{10}$$

where f_d is the doppler frequency of the *IR*. T_c is calculated to be around 3 ms which is much lesser than our designated time step of 0.1 s hence the system can be assumed to be a single tap flat fading channel.

The channel is modeled with Additive White Gaussian Noise (AWGN). The noise power is given by the equation (11)

$$N = N_0 \times \text{Bandwidth per RB} \times \text{Number of RBs} \tag{11}$$

6) Receiver: Each *IR* has a receiver RF chain that is capable of decoding the transmitted OFDM symbols and has a noise figure of 7 dB. At each receiver chain, the system removes the cyclic prefix, performs a 64-point FFT, equalizes the data with perfectly known channel state information. The receiver demaps the symbols to bits and performs the cyclic redundancy check to check if the bits have been correctly decoded. Based on the success of the CRC check, the base station provides actuation command to the *IR* according to the table IV.

CRC check	Actuation command	P_c	*IR* Action
Pass	`CONTINUE`	-	`CONTINUE`
Pass	`SLOW`	-	`SLOW`
Pass	`STOP`	-	`STOP`
Fail	-	$P_c > 0.5$	`STOP`
Fail	-	$P_c \leq 0.5$	`SLOW`

TABLE IV: Actuation command based on CRC success

Task	End time	Priority
Processing	32.5	2
Storage	94.6	2
Storage	125.7	2
Processing	156.5	2
Storage	185.5	2
Loading	219.1	2
Assembly	372	1
Loading	521.7	3
Assembly	522.4	1

(a) Tasks performed by IR_1

Task	End time	Priority
Loading	140.3	3
Storage	215.8	2
Assembly	281.8	1
Inspection	312.9	3
Loading	409.9	3
Loading	485.1	3
Assembly	522.4	1

(b) Tasks performed by IR_2

TABLE V: Tasks performed by each *IR* vs time step

III. Simulations

A. Base Implementation

The set-up is implemented for 522.4 s with 2 industrial robots for a heuristic resource block allocation algorithm. Our aim is to check the execution of the system to see the cyclic redundancy check success rate and if actuation commands are being correctly implemented in each *IR* in the running simulation. The tasks performed by each robot against the time steps is quantified in table V. Using the simulation run, we note the data from each *IR* at each 0.1 s time step which includes *IR* coordinates, inter *IR* distance, tasks being performed by each *IR* for the time step, collision probability between the *IR* pair P_c and the actuation command being sent to each *IR* at each time step. The simulation executes a close to collision event from time step `t = 457.9 s` to `t = 473.1 s` during which the probability of collision between the two *IRs* becomes non-zero.

B. Reinforcement Learning based resource allocation - 2 IRs

The heuristic RB allocation policy described in Section II allocates resource blocks proportional to task priority and applies a fixed binary boost when $P_c > 0.5$. The algorithm is computationally easy to execute but it is inherently static

and cannot adapt its collision-sensitivity to the current network state, nor learn from experience to trade off spectral efficiency against safety more precisely. We therefore replace the heuristic allocator with a contextual bandit based on the Upper Confidence Bound algorithm, which operates online and learns a state-dependent allocation policy directly from the reward signal at the base station.

1) Pipeline Modifications for Latency Distinction: A key observation motivates a necessary change to the PHY pipeline before introducing RL. In the base implementation, the downlink payload is only 10 bits (2-bit command + 8-bit CRC). With 7 resource blocks, each carrying 12 subcarriers, the entire payload is contained in a single OFDM symbol regardless of the number of resource blocks allocated. As a result, the per-*IR* latency is identical for all RB allocations, and the RL agent has no signal from which to learn the advantage of giving more RBs to a higher-priority robot.

To make the per-robot RB count consequential, the payload is extended to 160 bits (2-bit command + 8-bit CRC + 150 zero-padded bits), and the total RB pool is reduced to $N_{\text{RB}} = 7$. Under QPSK modulation, the 160-bit payload maps to 80 QAM symbols. With $n_{\text{SC}} = n_{\text{RB}} \times 12$ subcarriers per OFDM symbol, the number of OFDM symbols required is:

$$n_{\text{sym}} = \left\lceil \frac{80}{n_{\text{RB}} \times 12} \right\rceil \tag{12}$$

This creates a directly observable latency gap: a robot allocated 5 RBs (60 subcarriers) requires $\lceil 80/60 \rceil = 2$ symbols at QPSK, while one allocated 2 RBs (24 subcarriers) requires $\lceil 80/24 \rceil = 4$ symbols. Each OFDM symbol corresponds to $T_{\text{sym}} = 83.3$ μs, so the RL agent's RB decision directly determines the latency of each downlink actuation command. The transmit power is reduced to 10 mW so that Rayleigh deep fades can occasionally push the instantaneous SNR below the QPSK demodulation floor, producing realistic CRC failure rates of 6–9% at mid-factory distances. This gives the RL agent's channel quality component a meaningful signal to respond to.

2) UCB Contextual Bandit Formulation: The RL agent is modeled as a contextual UCB-1 bandit. At each time step t, the agent observes a state s_t summarizing the current network context, selects an action a_t from a discrete set, receives a scalar reward r_t, and updates its Q-value estimates using an incremental mean:

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}\big[r - Q(s,a)\big] \tag{13}$$

where $N(s,a)$ is the number of times action a has been taken in state s. Action selection follows the UCB-1 rule with exploration constant $c = 1.5$:

$$a_t = \arg\max_a \left[Q(s_t,a) + c\sqrt{\frac{\ln(N(s_t)+1)}{N(s_t,a)+1}} \right] \tag{14}$$

Unvisited (s,a) pairs are explored deterministically before any exploitation begins so the agent has complete context of the network without require an ϵ-decay schedule.

3) State Space: The agent's state s_t is a 3-tuple encoding the current collision risk, priority configuration, and channel quality:

$$s_t = \big(b_{\text{pc}},\ b_{\text{pp}},\ b_{\text{snr}}\big) \tag{15}$$

- $b_{\text{pc}} \in \{0,1,2,3\}$: four bins of the exponential moving average smoothed collision probability $P_{c,\text{smooth}}$, aligned to the MCS and command thresholds: safe (< 0.30), 16-QAM zone (< 0.50), approaching STOP (< 0.65), STOP zone (≥ 0.65).
- $b_{\text{pp}} \in \{0,\ldots,8\}$: nine values encoding the task priority pair (p_0, p_1) with each $p_i \in \{1,2,3\}$, as $b_{\text{pp}} = (p_0 - 1) \times 3 + (p_1 - 1)$.
- $b_{\text{snr}} \in \{0,1,2\}$: three SNR bins ($< 15$ dB / < 20 dB / ≥ 20 dB), corresponding to the QPSK, 16-QAM, and 64-QAM operating regimes respectively.

The total state space is $4 \times 9 \times 3 = 108$ states, which is small enough for the tabular UCB to achieve full coverage within a typical simulation run.

4) Action Space: The RL agent's action is the integer RB allocation for IR_0:

$$a_t = \text{rb}_0 \in \{1,2,3,4,5,6\}, \quad \text{rb}_1 = 7 - \text{rb}_0 \tag{16}$$

This gives 6 actions. The complementary allocation ensures that no RB is wasted and IR_1 always receives at least one RB. The heuristic policy, for reference, selects a split in $\{(5,2),\ (4,3),\ (3,4)\}$ deterministically based on priority and P_c.

5) Reward Function: The reward signal r_t is a weighted sum of five components, each targeting a distinct aspect of the system objective:

$$r_t = w_s R_{\text{safety}} + w_p R_{\text{priority}} + w_e R_{\text{spectral}} + w_c R_{\text{channel}} + w_l R_{\text{latency}} \tag{17}$$

with weights $w_s = 0.35$, $w_p = 0.25$, $w_e = 0.20$, $w_c = 0.12$, $w_l = 0.08$.

- **Safety** (R_{safety}): The primary component rewards low collision probability as $(1 - P_c)$. A hard penalty of -3.0 is applied when $P_c \geq 0.65$ (imminent collision) and an additional -1.5 if a CRC failure occurs while $P_c > 0.5$, since a lost actuation command during a high-risk interval is especially costly for the Q-function
- **Priority** (R_{priority}): Penalizes the absolute deviation of the actual RB fraction from the ideal priority-proportional fraction: $R_{\text{priority}} = -\left| \frac{\text{rb}_0}{7} - \frac{p_0}{p_0+p_1} \right|$. A bonus of $+0.5$ is awarded if the higher-priority *IR* achieves strictly lower latency than the lower-priority one, and a penalty of -0.5 otherwise
- **Spectral Efficiency** (R_{spectral}): Rewards total throughput as the fraction of the maximum achievable spectral efficiency:

$$R_{\text{spectral}} = \frac{\sum_i \text{bps}(\text{MCS}_i) \cdot \text{rb}_i \cdot 12}{6 \times 6 \times 12} \tag{18}$$

where $\text{bps}(\cdot)$ maps QPSK, 16-QAM, 64-QAM to 2, 4, 6 bits per subcarrier respectively

- **Channel Quality** (R_{channel}): Rewards successful CRC decoding ($+0.5$ per robot), with a bonus for using 64-QAM when the average SNR supports it, and a penalty for unnecessary QPSK when the channel is clean ($P_c < 0.30$)
- **Latency/Deadlock** (R_{latency}): Penalizes prolonged simultaneous stopping of both *IRs* (deadlock), as a ratio of the per-robot stop counter to the maximum allowed stop duration

6) Limitations: The fundamental challenge with the 2-*IR* formulation is that the safety-critical states occur with very low probability. Since there is only one robot pair navigating a 200 m $\times$ 200 m floor, the smoothed collision probability $P_{c,\text{smooth}}$ exceeds the threshold for only a small fraction of total simulation time. As a result, the UCB agent is able to accumulate only very limited updates for the high-risk bins ($b_{\text{pc}} \in \{2, 3\}$), and the Q-values in those states cannot converge. The agent can learn a good policy for the normal operating regime but remains largely unlearned precisely where reliable collision-avoidance allocation matters most. This motivates us to extend the simulation to 5 *IRs* so our Q-function values can converge and the RL agent performs as intended.

C. Reinforcement Learning based resource allocation - 5 IRs

With five *IRs* operating simultaneously, the number of robot pairs that the base station must monitor is $\binom{5}{2} = 10$, compared to just one pair in the 2-*IR* case. The per-robot smoothed collision probability is defined as the maximum over all *IR* pairs involving that robot:

$$P_{c,i} = \max_{j \neq i} P_{c,\text{smooth}}^{(i,j)} \tag{19}$$

Because any one of ten pairs can independently trigger a high-risk event, the probability that at least one robot pair is in a collision-critical state at any given time step is substantially higher than in the 2-*IR* scenario. This exposes the RL agent to safety-critical states with sufficient frequency for the UCB to accumulate a large number of Q-value estimates and learn a collision-aware allocation policy.

1) Scaling resource block pool: To maintain a fair comparison with the 2-*IR* case, the total resource block pool is scaled to preserve the same average per robot allocation. In the 2-*IR* system, each robot receives on average $7/2 = 3.5$ RBs. Extending this to five robots gives:

$$N_{\text{RB}}^{(5)} = \text{round}(3.5 \times 5) = 17 \text{ RBs} \tag{20}$$

yielding an average of $17/5 = 3.4$ RBs per robot. The same limit of 5 resource blocks (60 subcarriers, within the 64-point FFT constraint) per *IR* and a minimum of 1 RB are enforced. Since n_{sym} depends only on the number of RBs per *IR* and not on the total pool size, the latency table is identical for the two cases. A robot allocated 5 RBs under QPSK requires 2 symbols (166.7 μs) and one allocated 2 RBs requires 4 symbols (333.3 μs). All other PHY parameters including payload size, MCS thresholds, TX power, channel model are kept identical to the 2 *IR* pipeline, ensuring that performance comparisons are on the same platform.

2) Score based Proportional RB Allocation: With five robots, the 2 *IR* heuristic which selected a fixed split based on priority and a binary collision threshold does not generalize. Instead, each robot is assigned a score that jointly encodes its task priority and its current collision risk as

$$\sigma_i = p_i \times (1 + \alpha \cdot P_{c,i}) \tag{21}$$

where $p_i \in \{1, 2, 3\}$ is the task priority of IR_i and $\alpha \geq 0$ is the collision weight parameter. The integer RB allocation is then computed as

$$\tilde{n}_i = \frac{\sigma_i}{\sum_j \sigma_j} \times N_{\text{RB}}^{(5)}, \quad n_i = \text{clamp}(\lfloor \tilde{n}_i \rfloor, 1, 5) \tag{22}$$

with fractional remainders used to distribute any deficit such that $\sum_i n_i = 17$ exactly. In the heuristic baseline, α is fixed at $\alpha = 1.0$. When $\alpha = 0$, allocation is purely priority-proportional; larger α increasingly concentrates resources on the robots facing the highest collision risk.

Deadlock Resolution With five *IRs*, the scenario where multiple robots simultaneously receive a `STOP` command—a deadlock—is substantially more likely than in the 2 *IR* case. To prevent factory-wide gridlock, a deadlock resolver is applied after individual command selection: If three or more robots receive `STOP` at the same time step, the two lowest-priority stopped robots remain halted (yielding right of way), while the remaining stopped robots are upgraded to `SLOW` actuation command. This ensures that the highest-priority *IRs* are never fully blocked by a collision resolution procedure.

3) RL Formulation: The main challenge in extending the RL framework to five robots is the action space. Directly allocating integer RBs to five robots produces $\binom{17-1}{5-1} = 4{,}368$ feasible distributions which is far too many for a tabular UCB agent to explore in reasonable time. We resolve this by restructuring the action space. Instead of selecting the allocation directly, the RL agent selects the collision weight α and the score-based formula above deterministically converts this choice into a specific RB distribution. The heuristic policy corresponds to $\alpha = 1.0$; the RL agent must learn whether to be more conservative ($\alpha < 1.0$, emphasizing spectral efficiency) or more aggressive ($\alpha > 1.0$, concentrating RBs on collision-endangered robots).

The discrete action set is

$$\alpha \in \{0.0,\ 0.5,\ 1.0,\ 1.5,\ 2.0,\ 3.0\} \tag{23}$$

This gives 6 actions, the same cardinality as the 2 *IR* case (which had 6 choices of rb_0), enabling a direct comparison of convergence behavior between the two formulations.

4) State Space: The state for the 5-IR agent is a 4-tuple that captures the global collision-risk landscape, priority configuration, channel quality and the simultaneous risk intensity:

$$s_t = \left(b_{\text{pc}},\ b_{\text{pp}},\ b_{\text{snr}},\ b_{\text{risk}}\right) \tag{24}$$

- $b_{\text{pc}} \in \{0, 1, 2, 3\}$: same four bins as the 2 *IR* case, applied to the global maximum $P_{c,\text{smooth}}$ across all 10 pairs
- $b_{\text{pp}} \in \{0, \ldots, 8\}$: the $(p_{\text{hi}}, p_{\text{lo}})$ encoding of the *highest-risk pair* the pair (i, j) achieving the maximum $P_{c,\text{smooth}}^{(i,j)}$.

This encoding is identical to the 2 *IR* priority pair index, preserving direct comparability between the two state spaces

- $b_{\text{snr}} \in \{0, 1, 2\}$: average SNR bin across all five *IRs*, using the same thresholds as the 2 *IR* case
- $b_{\text{risk}} \in \{0, 1, 2\}$: the number of pairs with $P_{c,\text{smooth}} > 0.5$, binned as zero, one, or two-or-more. This dimension is unique to the 5-IR case and gives the agent information about how many pairs are simultaneously at risk, i.e., the scenario where a high α is most beneficial

The total state space has $4 \times 9 \times 3 \times 3 = 324$ states. This is larger than the 2 *IR* case (108 states) but remains tractable for the tabular UCB, since the 5 *IR* scenario visits high-risk states ($b_{\text{pc}} \geq 2$, $b_{\text{risk}} \geq 1$) with sufficient frequency to populate those entries.

5) Execution of the RL algorithm in simulation: The key insight of the 5 *IR* RL design is that the optimal α is dependent and adaptable to the context of the environment in a way that a fixed heuristic cannot capture. When the global collision probability is low and no pairs are at risk ($b_{\text{pc}} = 0$, $b_{\text{risk}} = 0$), the agent should favor $\alpha = 0$ (pure priority allocation) to maximize spectral efficiency. When a single high-priority pair is at risk ($b_{\text{pc}} \geq 2$, $b_{\text{risk}} = 1$), a moderate α concentrates extra RBs on that pair without disrupting the rest. When multiple pairs are simultaneously endangered ($b_{\text{risk}} = 2$), the highest α values (2.0 or 3.0) ensure that the most collision-exposed *IRs* receive the RBs needed for robust, low-latency actuation command delivery. The UCB agent, having observed outcomes across all 324 states, learns these context-dependent trade-offs in an experimental manner which cannot be achieved by the fixed-α heuristic algorithm.

The reward function for the 5 *IR* case uses the same five components and identical weights as the 2 *IR* formulation, with the following generalizations:

- R_{safety} is computed from the global maximum $P_{c,\text{smooth}}$
- R_{priority} measures the mean absolute deviation of actual RB fractions from ideal priority fractions across all five robots
- R_{spectral} sums spectral efficiency over all robots normalized by the maximum achievable $6 \times 5 \times 12 \times 5 = 1800$ SC-bps
- and R_{latency} penalizes the mean stop-counter ratio across all five robots

IV. Results

In this section, we present some results for the base implementation, depicting the network throughput and IR velocities over time. The latter validates the end-to-end system performance and the correct decoding of actuator command at the IR, and the former shows how the throughput is traded off for robustness in the case of collision.

A. IR Velocity Over Time - Collision Visualization

We observe in Figure 3, that most of the time the robots move at a constant velocity of 3m/s upon the actuation command `CONTINUE`, since no collision risk is estimated.

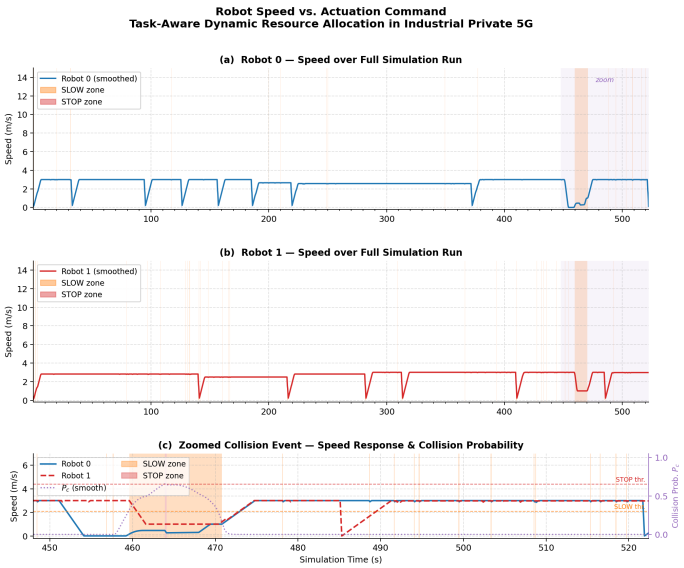


Fig. 3: IR velocity over time for different downlink commands

The periodic stops in the time series indicates the workstation stops, which is the expected routing behaviour. At `t = 448 s`, a collision event is detected and both IRs slow down. IR_0 almost comes to a stop, and IR_1 slows to a constant 1m/s, since it had higher task priority and hence the right of way. As the IRs slowly move past each other, the collision probability reduces and the speeds gradually recover. This validates our sensing-to-action pipeline and depicts successful decoding of the actuation command at the IR receivers.

In figure 3, Part (c) zooms in on the collision event that occurred at `t = 448–522 s`. The blue curve indicates the velocity for IR_0, and the red curve indicates the velocity for IR_1. The light purple dotted curve shows the collision probability over this time interval. We observe that as the P_c rises over 0.3, both the robots slow down. Looking at the dotted yellow and red horizontal lines, which represent the `SLOW` and `STOP` thresholds, we see that P_c never crosses the `STOP` threshold of 0.65, and hence none of the robots actually stop, resulting in efficient collision prevention.

B. Network Throughput Over Time

We plot the network throughput over time and map out the regions of collision probability where the MCS changed from 64-QAM to 16-QAM to QPSK in the figure 4. Both IRs receive with 64-QAM without collision. During the collision, we can notice that the throughput drops to 16-QAM first and then to QPSK as the IRs get increasingly closer. Observing the zoomed in view in part (b), during the times when $0.3 \leq P_c \leq 0.6$, the throughput drops to the 16-QAM level of 5.04kbps, since the command is `SLOW`. Subsequently, as the robots get closer still, MCS switches to QPSK level as $P_c \geq 0.6$, for robust actuation. The light yellow zones indicate the 16-QAM regions, and the red zone in the middle is the QPSK region.

The brief vertical drops in the network throughput indicate the CRC failures which prevent decoding at that time step. The

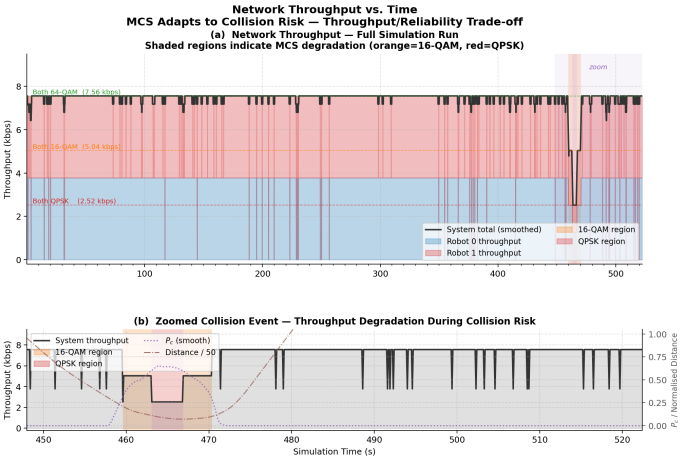


Fig. 4: Network Throughput Over Time

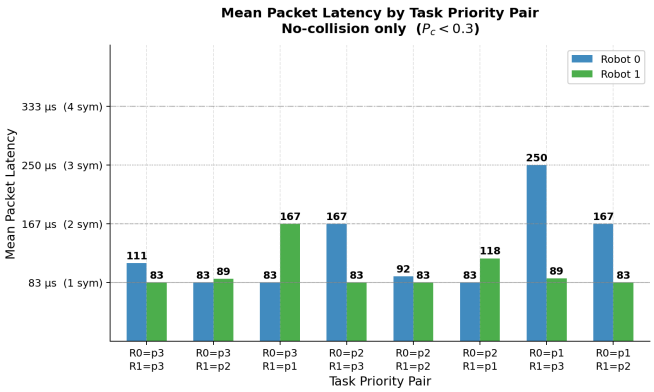


Fig. 5: Mean Packet Latency by Task Priority Pair under normal operating conditions ($P_c < 0.3$).

overall throughput delivered to the system is largely optimized to the 64-QAM level, and the CRC pass rate which is 98.4%. Deployment of more IRs in the same system would cause more dips in throughput across time, reducing the average throughput of the system, but ensuring collision prevention.

C. Priority-Aware Latency Differentiation

To understand the impact of task-aware scheduling, we evaluate the mean packet latency by task priority pair in the absence of collision risks ($P_c < 0.3$). As shown in Figure 5, the higher-priority robot consistently receives a larger share of the resource blocks. This increased RB allocation translates to the payload fitting into fewer OFDM symbols, thereby reducing the overall packet latency. For instance, a robot executing the highest priority task ($p = 3$) achieves a minimal latency of 83 μs, corresponding to a single OFDM symbol. In contrast, lower priority tasks (e.g., $p = 1$) incur higher latencies ranging between 167 μs and 333 μs. This demonstrates that the latency differentiation in normal operation is strictly a function of the RB allocation algorithm.

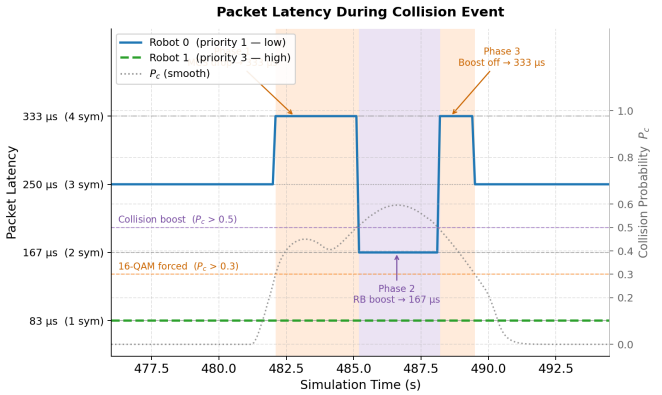


Fig. 6: Packet Latency variations during a collision event for a high-priority and low-priority robot.

D. Packet Latency During a Collision Event

The dynamic nature of the resource allocation is highlighted during a collision event, where the base station trades spectral efficiency for robustness and latency adjustments. Figure 6 traces the packet latency through three distinct phases of a close encounter. In Phase 1, as the collision probability rises above the 0.3 threshold ($P_c > 0.3$), the network forces a reduction in the MCS to 16-QAM for robustness, which inadvertently causes the low-priority robot's latency to spike to 333 μs. In Phase 2, as the risk becomes critical ($P_c > 0.5$), the RB collision boost activates; this reallocates resources and drops the latency back to 167 μs to ensure timely delivery of safety commands. Finally, in Phase 3, the boost deactivates as the robots pass each other, temporarily returning the latency to 333 μs before fully recovering to baseline. Notably, the high-priority robot ($p = 3$) maintains a constant 83 μs latency throughout the entire event, remaining completely unaffected by the network's reallocation maneuvers.

E. Productivity Cost of Collision Avoidance

While the collision avoidance mechanisms successfully ensure safety, they introduce a quantifiable cost to overall factory throughput. We measure this using the Workstation Productivity and Fairness Index (WPFI), defined as the priority-weighted workstation visits calculated over a rolling 60-second window. Figure 7(a) illustrates how the collision zones (indicated by orange and purple shading) directly delay the subsequent arrival spikes at workstations, temporarily driving the WPFI downward. Panel (b) quantifies this cumulative productivity loss by comparing the actual cumulative WPFI against an ideal baseline where no collisions occur. The widening gap between the baseline and the actual performance represents the systemic productivity cost incurred by the necessary collision avoidance decelerations.

F. Performance with higher number of IRs

Figure 8 shows a comparison of the 5 *IR* and 2 *IR* UCB agents evaluated on their respective simulation runs (1712 s and 1136 s at 0.1 s step resolution).

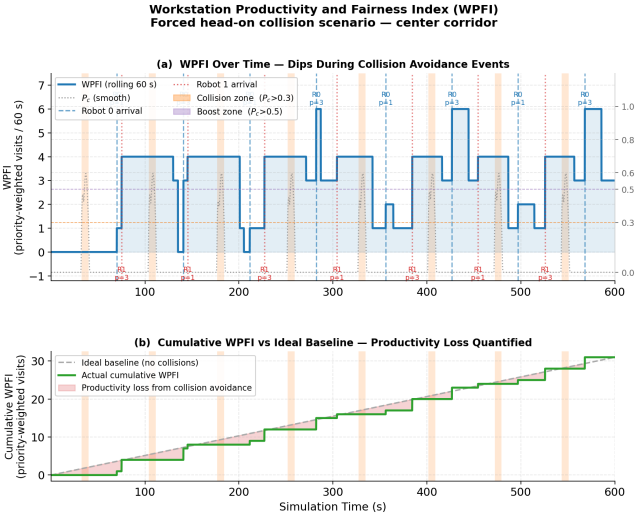


Fig. 7: Workstation Productivity and Fairness Index (WPFI) demonstrating the productivity cost of collision avoidance.

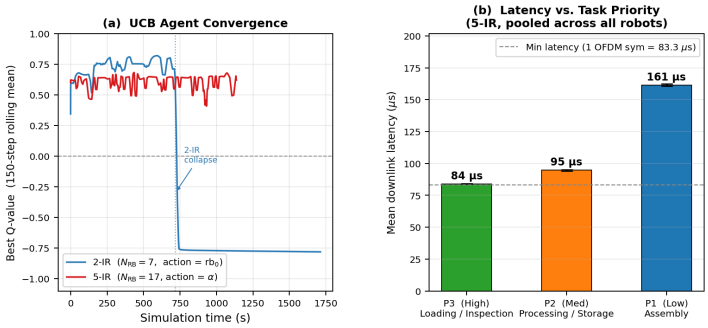


Fig. 8: Comparison of the 2 *IR* and *5* UCB-RL agents. (a) Best Q-value over time (150-step rolling mean). (b) Mean downlink latency stratified by task priority in the 5-IR case, pooled across all five *IRs*.

Fig (a) shows the evolution of the UCB agent's best Q-value over simulation time. The 2 *IR* agent initially learns a positive policy (peak $\approx +0.82$), but collapses at $t = 717$ s when both robots enter a proximity lock at inter-robot distance $\approx$ 4 m, driving $P_{c,\text{smooth}}$ permanently above 0.65 for the remainder of the run. The Q-value degrades to 0.78 and never recovers, since the UCB agent continues exploiting a policy that was learned under a fundamentally different operating regime. The 5 *IR* agent by contrast maintains a stable Q-value of approximately $+0.60$ throughout the entire run with low variance, demonstrating that the richer multi-pair collision environment prevents the agent from locking into a pathological state.

Fig (b) shows the mean downlink latency stratified by task priority in the 5 *IR* case, pooled across all five *IRs* and over 56,320 robot-steps. The result is a clean monotonic ordering - P3 (high-priority) robots receive mean latency of 84 μs within 1 μs of the physical minimum of one OFDM symbol, while P1 (low-priority) robots receive 161 μs, a gap of 77 μs. P2 robots fall between the two at 95 μs. This ordering holds for every individual robot in the simulation. The result confirms that the score-based RB allocation, even with a non-converged α, correctly channels more resource blocks to higher-priority *IRs*, reducing their n_{sym} and thereby their actuation command latency. This is precisely the task-aware behaviour that the system was designed to enforce.

V. Conclusion

This report presents the implementation of an industrial private 5G network with dynamic resource allocation for collision prevention and task-aware IR operation. The results show that our heuristic resource allocation model achieved higher throughput for IRs performing higher priority tasks. The dynamic MCS selection successfully avoided multiple collision risks due to timely switching to more robust modulation. Moreover, IR velocity per time slot corroborates with the actuator commands sent in the downlink, verifying that the control bits are being decoded correctly over a Rayleigh fading channel, AWGN noise, and an industrial path loss model. The use of the RL algorithm using UCB demonstrated a generalized performance to 5 IRs by taking into account multi-IR collision, priorities and SNR in order to calculate the reward functions.

Acknowledgment

We would like to extend our gratitude to Prof. Dinesh Bharadia for giving us the opportunity to explore research related to the problems faced by the wireless communication and networks industry and providing us with support throughout the project duration. We would also like to acknowledge the resources provided by the teaching assistants of the course which were crucial to the execution and analysis of our experiments.

References

[1] F. Al-Tam, N. Correia, and J. Rodriguez, "Learn to Schedule (LEASCH): A Deep Reinforcement Learning Approach for Radio Resource Scheduling in the 5G MAC Layer," *IEEE Access*, vol. 8, pp. 108088–108101, 2020. doi: 10.1109/ACCESS.2020.3000893.

[2] J. Ma, A. Aijaz, and M. Beach, "Recent Results on Proportional Fair Scheduling for mmWave-based Industrial Wireless Networks," in *Proc. IEEE 92nd Vehicular Technology Conference (VTC2020-Fall)*, 2020, pp. 1–5. doi: 10.1109/VTC2020-Fall49728.2020.9348753.

[3] R. Wang, H. Beshley, L. Yan, O. Urikova, M. Beshley, and O. Kuzmin, "Industrial 5G Private Network: Architectures, Resource Management, Challenges, and Future Directions," in *Proc. IEEE 16th Int. Conf. on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, 2022, pp. 780–784. doi: 10.1109/TCSET55632.2022.9766945.

[4] M. Wen, Q. Li, K. J. Kim, D. López-Pérez, O. A. Dobre, H. V. Poor, P. Popovski, and T. A. Tsiftsis, "Private 5G Networks: Concepts, Architectures, and Research Landscape," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 1, pp. 7–25, 2022. doi: 10.1109/JSTSP.2021.3137669.

[5] A. Mathur and A. Bharadwaj, "Physical Resource Block Allocation in Private 5G Network for Handoff-Free Operations," in *Proc. 17th Int. Conf. on Communication Systems and Networks (COMSNETS)*, 2025, pp. 685–693. doi: 10.1109/COMSNETS63942.2025.10885666.

[6] M. A. Lema, A. Laya, T. Mahmoodi, M. Dohler, and M. Shariat, "Private 5G Networks: A Survey on Enabling Technologies, Deployment Models, Use Cases and Research Directions," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2588–2630, 2021. doi: 10.1109/COMST.2021.3097445.